

Semester: Fall 2026–2027

Course Code and Name: BACSE202 – Database Management Systems

Faculty Name: Dr.S.Geetha

Project title : Cloud Sphere

Team members' names and register numbers:

Reg No	Name
25BCE5776	Rohith V S
25BCE5350	Dharaneeshwaran
25BCE1991	Raakesh M

GitHub repository link: https://github.com/DHARANEESWARAN08/CloudSphere

YouTube demo video link: https://youtu.be/pPAymDPT_og?si=2hbXw9faf10FEIbx

Abstract

CloudSphere is a database-backed academic simulator for managing organizations, projects, virtual machine requests, and shared infrastructure capacity. Customers can request a VM and review its estimated cost; administrators can approve or reject the request. PostgreSQL routines validate quota, budget, and host capacity, record the simulated VM allocation, and preserve lifecycle and audit history. The application combines a React and TypeScript interface, an Express API, and PostgreSQL. Its relational schema uses keys, constraints, indexes, triggers, transactions, and time-partitioned monitoring records. Seed data and automated tests support repeatable demonstrations. CloudSphere does not launch a real virtual machine or process a real payment, so it is presented as a database and workflow simulator rather than a cloud provider.

Keywords: cloud resource management, PostgreSQL, relational database, VM request approval, resource quota, transaction safety

Report Contents

- 1 Introduction
- 2 Existing Systems
- 3 Requirements and System Architecture
- 4 ER Diagram and Relational Schema
- 5 Keys Constraints and Normalization
- 6 Implementation and SQL Operations
- 7 Testing Results and Discussion
- 8 Application Screenshots
- 9 Conclusion Limitations and Future Work
- References

1 Introduction

Problem Statement

Cloud infrastructure teams need a consistent way to organize projects, review compute requests, enforce resource limits, and trace allocation decisions. A dashboard that changes only its display without enforcing those rules would be unreliable. CloudSphere addresses the data-management part of this problem by keeping requests, quotas, hosts,

VM state, and audit records in PostgreSQL and exposing controlled actions through an application API.

Objectives

- Organize users, organizations, projects, plans, regions, hosts, and VM configurations in a relational schema.
- Support cost estimation and an administrator approval step before a request allocates resources.
- Enforce quota, budget, and capacity checks in database routines with transaction and row-lock protection.
- Provide role-specific application screens and preserve VM lifecycle, notification, and audit history.
- Demonstrate database design and implementation practices through migrations, seed data, queries, and automated test suites.

Scope

The project covers an academic simulation of cloud-management workflows. PostgreSQL stores and validates simulated infrastructure records. The frontend and API allow customer and administrator interactions. Real hypervisor provisioning, operating-system boot, SSH access, live workload metrics, production billing, and payment processing are outside the implemented scope. Monitoring, billing, storage, networking, SLA, carbon, spot-market, and recommendation records should be described as application data or simulated views unless a feature is explicitly connected to a real service.

2 Existing Systems

Amazon EC2 and OpenStack Horizon provide established interfaces for managing compute instances. Their documentation describes project or account views, instance state, launch workflows, and administrative controls [8], [9]. They operate real cloud infrastructure. CloudSphere uses related resource-management concepts for a database course demonstration, but its VM records do not represent running cloud hardware.

System	Documented capabilities	Relevance to CloudSphere
Amazon EC2 Console	Lists instances and provides actions to launch, inspect, and troubleshoot instances [8].	Reference for inventory and lifecycle workflows; CloudSphere only simulates VM state.
OpenStack Horizon	Project users manage instances, images, key pairs, and related resources; admins have separate resource controls [9].	Reference for role separation, project scope, and user/admin views.

System	Documented capabilities	Relevance to CloudSphere
CloudSphere	Customer and admin workflows for projects, VM requests, capacity, quotas, notifications, and audit records [1].	Course implementation focused on relational integrity and transaction-safe allocation.

The repository contains three screenshots of the CloudSphere implementation. They appear in Section 8. Screenshots of AWS or OpenStack accounts were not included; their official documentation is linked in References for comparison.

3 Requirements and System Architecture

Users and Functional Requirements

User	Implemented workflow
Customer	Sign in; view organization projects; inspect VM records; estimate a VM request; submit it for admin review; receive decision notifications; use supported VM lifecycle actions.
Administrator	Review the VM request queue; approve or reject requests with a reason; inspect host capacity and audit records; manage quotas and catalog data.
Database/API	Enforce role and organization scope; validate resource limits; serialize allocation with row locks; record lifecycle changes and notifications.

Quality and Data Requirements

- Use foreign keys and checks to reject invalid or orphaned data.
- Keep application database permissions narrower than the migration role; lifecycle and allocation counters are controlled through database routines.
- Use parameterized SQL from the API and avoid returning password hashes or secrets.
- Provide repeatable local setup through numbered SQL migrations and deterministic seed data.
- Use only synthetic development identities and data in demonstrations.

Proposed Architecture

The browser runs the React and TypeScript client. The client sends credentialed REST requests to the Express API, which authenticates the user, validates inputs, and queries PostgreSQL through the pg driver. PostgreSQL constraints, stored procedures, triggers, and transactions protect shared state. No hypervisor or external cloud API is part of the flow.

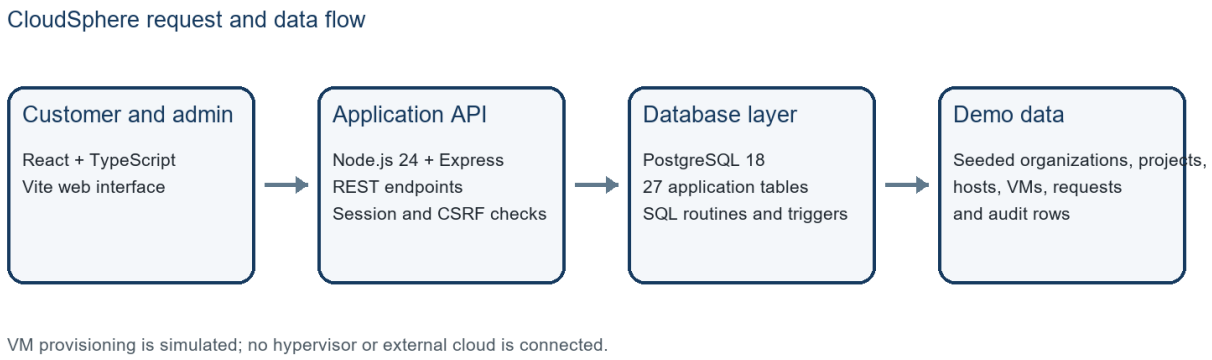
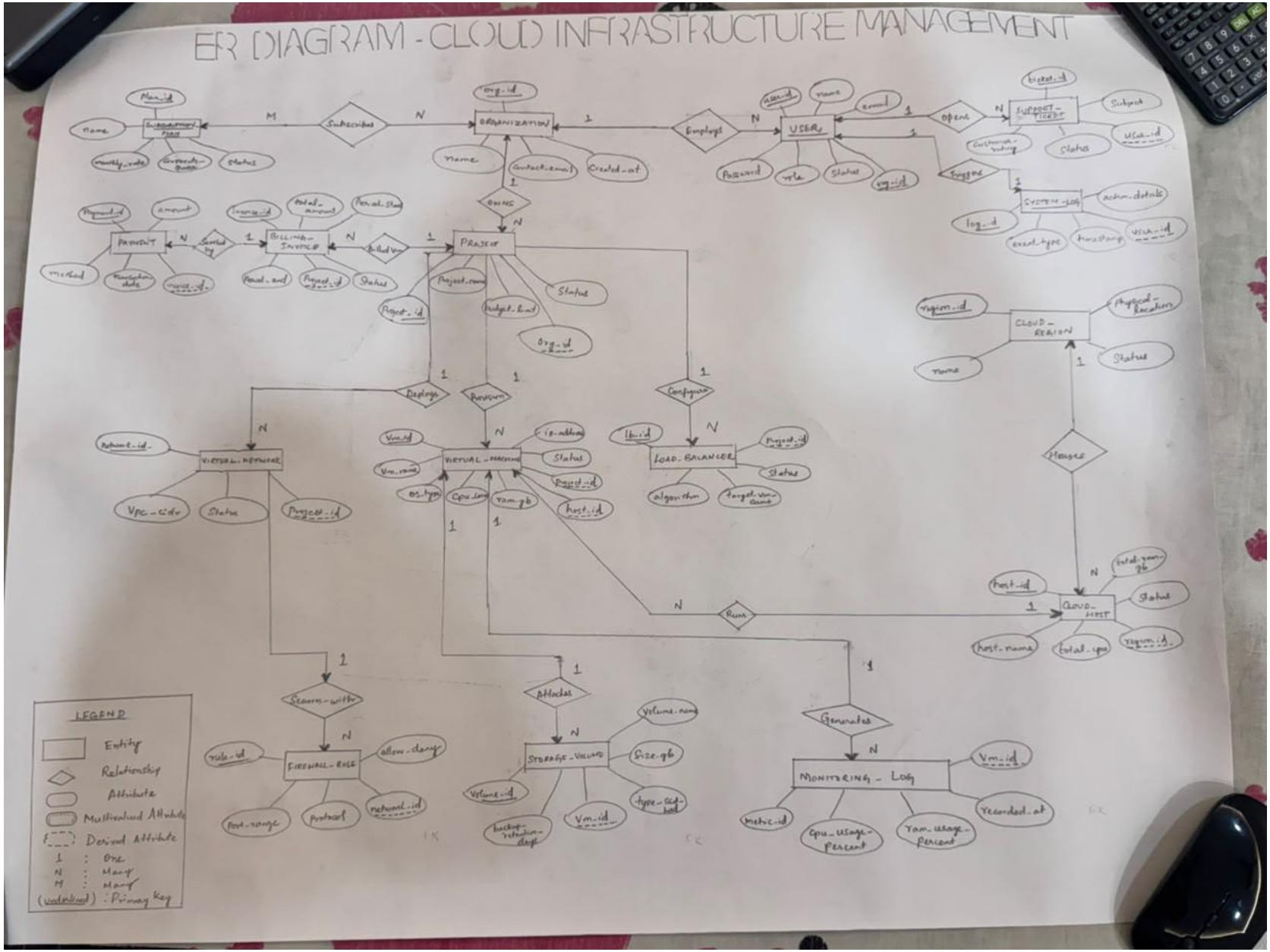
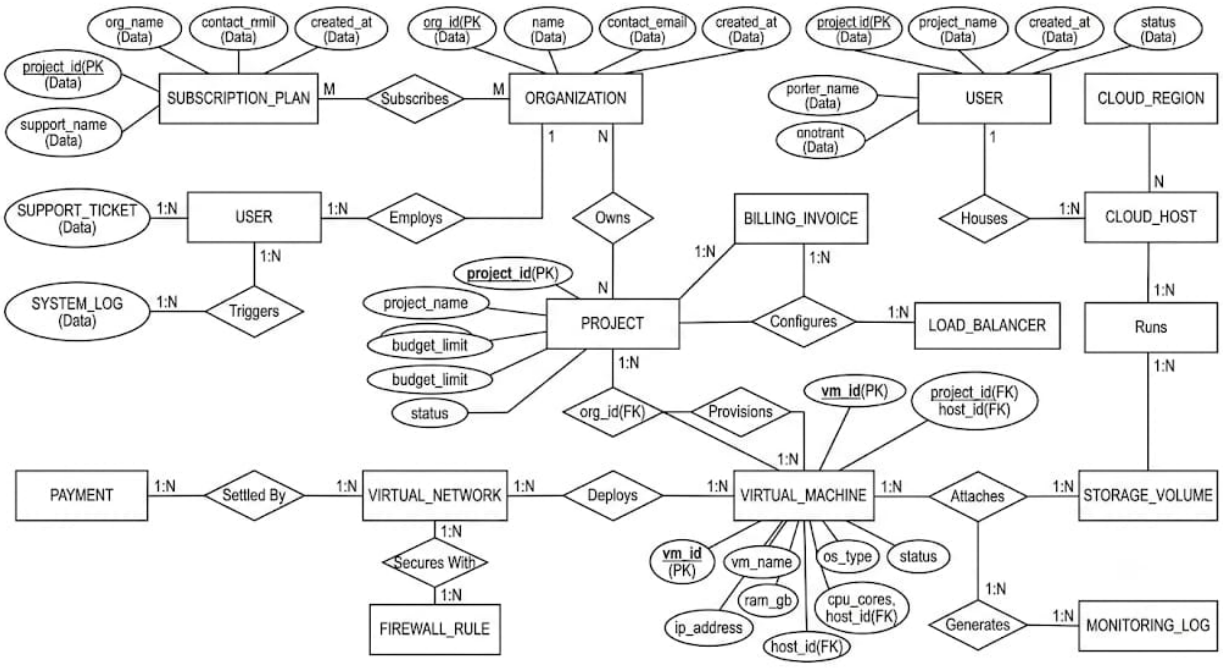


Figure 1. CloudSphere application and database architecture.

4. ER Diagram and Relational Schema


ER DIAGRAM - CLOUD INFRASTRUCTURE MANAGEMENT
SUBSCRIPTION PLAN
SUBSCRIBES
ORGANIZATION
EMPLOYS
USERS
OPENS
SUPPORT TICKET
OWNS
PAYMENT
BILLING INVOICE
PROJECT
SYSTEM LOG
CLOUD REGION
DEPLOYS
PERFORMS
CONFIGURES
HOSTED
VIRTUAL NETWORK
VIRTUAL MACHINE
LOAD BALANCER
RUNS
CLOUD HOST
SECURES WITH
FIREWALL RULE
ATTACHES
STORAGE VOLUME
GENERATES
MONITORING LOG
LEGEND
Entity
Relationship
Attribute
Multivalued Attribute
Derived Attribute
1 : One
N : Many
M : Many
(underline) : Primary key



5 Keys Constraints and Normalization

Keys and Integrity Constraints

Database mechanism	Example in the implementation
Primary keys	Identity keys identify plans, organizations, projects, users, hosts, VM requests, and VMs.
Foreign keys	Projects reference organizations; requests reference projects and requesters; VMs reference projects, configurations, and optional hosts.
Candidate and unique keys	Lower-cased user email is unique; project names are unique within an organization; VM configuration is unique by vCPU, RAM, and OS image.
CHECK constraints	Restrict resource values and lifecycle states; prevent host allocation from exceeding totals; validate request status consistency.

Database mechanism	Example in the implementation
Generated columns and partial indexes	Host free capacity is total minus allocated capacity; partial unique indexes prevent duplicate active names or pending request names.
Triggers and procedures	Validate state transitions, update timestamps, record state changes, notify on low capacity, and manage reservations.

Functional Dependencies

Representative functional dependencies follow from declared primary and candidate keys:

- plan_id → plan name, unit prices, SLA target, and default limits.
- organization_id → organization name, subscription plan, billing email, and status.
- project_id → organization_id, project name, budget, and status.
- vm_configuration_id → vCPU, RAM, OS family, and OS image; the configuration tuple (vCPU, RAM, OS image) is also unique.
- vm_id → project_id, configuration, host, VM name, and current state.
- host_id → region_id, hostname, total capacity, and allocated capacity.

Normalization to Third Normal Form

A wide VM report could repeat project names, organization details, plan prices, configuration values, host details, and region names for every VM. The schema separates those entities and stores their keys in dependent relations. For example, virtual_machine stores project_id, vm_configuration_id, and host_id; it does not repeat the project name, organization plan, CPU/RAM catalog data, or host region. Non-key attributes in these core relations are stored with the entity key that determines them. This decomposition removes repeating groups and avoids partial or transitive dependencies in the core records, supporting at least 3NF for these relations.

Some counters are intentionally stored for allocation safety and quick checks: cloud_host.allocated_cpu/allocated_ram_gb and resource_quota.used_* are updated transactionally and reconciled in tests. JSON snapshots preserve values recorded in audit and usage history. These are documented implementation choices rather than accidental copies of current master data.

6 Implementation and SQL Operations

Technology Stack

Layer	Technology and role
Frontend	React 19, TypeScript, Vite, Bootstrap, TanStack Query, Axios, and Recharts.
Backend	Node.js 24, Express 5, pg, Zod, Helmet, CORS, and HTTP-only session cookies.
Database	PostgreSQL 18 in the demonstrated setup; PL/pgSQL migrations, constraints, routines, triggers, generated columns, indexes, and range partitions.
Setup and evidence	Numbered migrations, seed script, environment examples, API and database test suites, and pgAdmin/psql guides.

Request Approval and Allocation Flow

- The customer chooses a project and VM details, and the API returns a cost estimate.
- Submitting the form creates a PENDING row in vm_request; a pending request reserves no host capacity.
- An administrator approves or rejects the request. Rejection stores the decision reason.
- Approval calls allocate_resource, which checks access, budget, quota, and host capacity while locking relevant rows. It creates the simulated VM and related storage/allocation records, then completes simulated provisioning.
- The decision and state change are recorded, and the requester can receive an in-app notification.

Example Read Query

This query joins VM, project, and configuration data to display the resource configuration of running VM records.

```
SELECT vm.name AS vm_name, p.name AS project_name,
       cfg.vcpu, cfg.ram_gb, vm.state
FROM virtual_machine AS vm
JOIN project AS p
  ON p.project_id = vm.project_id
JOIN vm_configuration AS cfg
  ON cfg.vm_configuration_id = vm.vm_configuration_id
WHERE vm.state = 'RUNNING'
ORDER BY p.name, vm.name;
```

The joins use foreign-key identifiers. The WHERE clause filters to records marked RUNNING; it does not show that a real operating system is running.

Database Connectivity and Security

The backend reads DATABASE_URL and JWT_SECRET from its environment. The frontend uses VITE_API_BASE_URL to reach the API; database credentials must remain in the backend environment. The API uses role checks, organization-scoped queries, parameterized SQL, CSRF validation for state-changing requests, and secure cookie settings appropriate to local HTTP or HTTPS. The repository provides separate migration and application roles, with allocator-owned fields managed through granted routines.

7 Testing Results and Discussion

The repository includes Node test suites for allocation, API behavior, concurrency, VM lifecycle, monitoring partitions, database privileges, and seed reconciliation [1]. The tests are designed to use a separate database whose name ends in _test; helpers refuse to reset a database without that suffix. This report preparation did not execute the test suite, so it does not claim passing results.

Test area	Expected result in test source	Execution status
Allocation and quota	Valid requests reserve host/quota capacity; over-limit requests are rejected without counter drift.	Not run during report preparation.
Concurrency	Concurrent allocations cannot over-allocate host or quota; counters reconcile.	Not run during report preparation.
Lifecycle	Allowed transitions succeed; repeated release does not subtract capacity twice.	Not run during report preparation.
API and access control	Anonymous requests are rejected; signed-in users see only authorized organization data.	Not run during report preparation.
Partitions and privileges	Time queries prune partitions; app role cannot directly edit allocator counters or VM state.	Not run during report preparation.
Seed consistency	Seeded host/quota counters match VM rows and associated resources.	Not run during report preparation.

Demonstration Evidence

The repository README contains three application screenshots. Test source provides behavior-level assertions, but no test-run log or pgAdmin result screenshot was supplied for this report. Before submission, run the suite against the isolated test database, record actual pass/fail counts, and capture a database result that matches a demonstrated UI workflow.

8 Application Screenshots

These images are committed in the project documentation and are described in the README as screenshots captured from the local demonstration application [1].

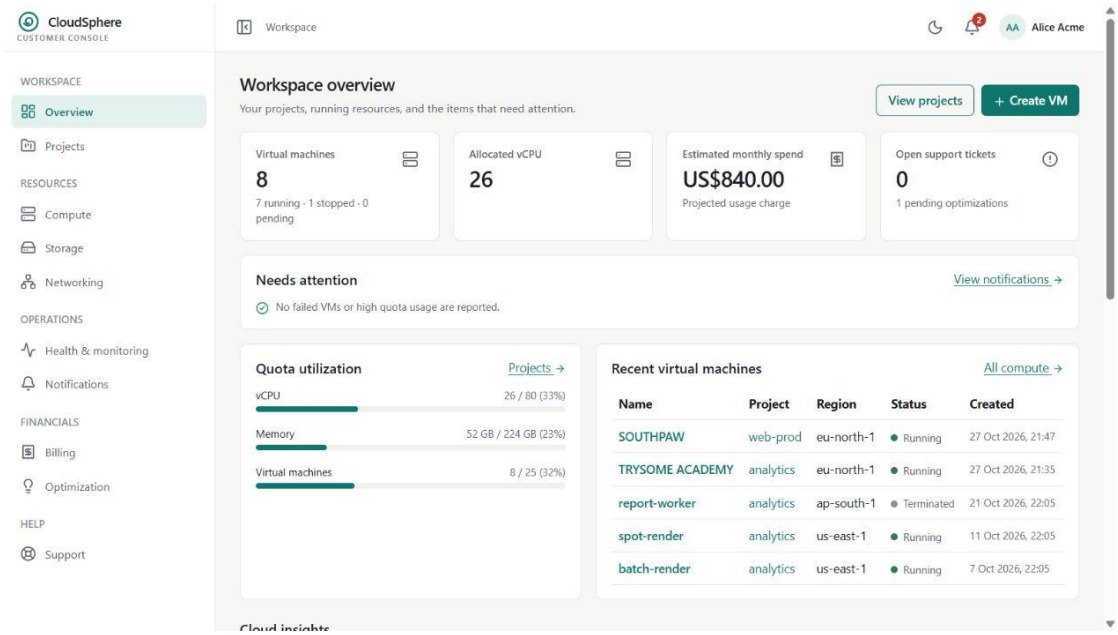


Figure 3. Customer workspace summary with VM status, resource usage, estimates, and quota information.

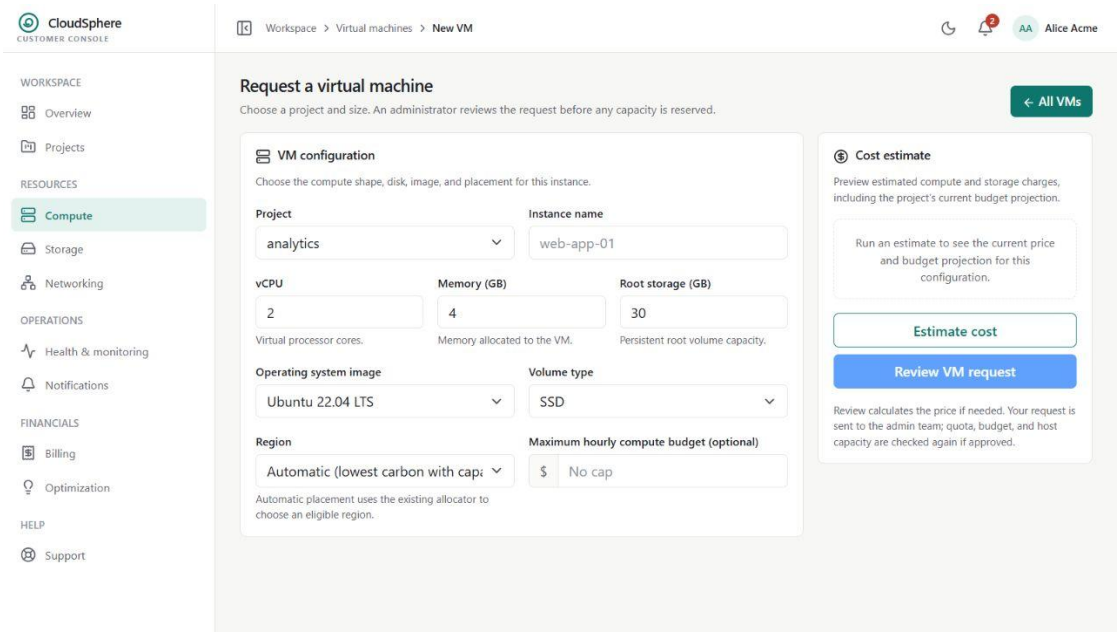


Figure 4. VM request form with configuration choices and the cost estimate panel.

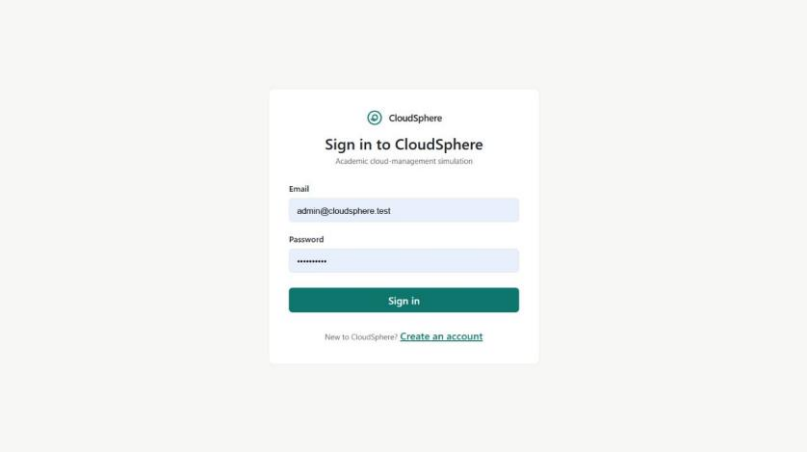


Figure 5. Shared sign-in screen for customer and administrator demo accounts.

9 Conclusion Limitations and Future Work

CloudSphere demonstrates a relational database-backed workflow for project scoping, VM requests, administrator decisions, simulated resource allocation, and auditability. Its strongest DBMS contribution is the use of relational constraints and transaction-safe

routines to keep simulated quota and host counters consistent as users request and manage VM records.

Limitations

- VM provisioning, start/stop, and termination update simulated database records; there is no hypervisor integration or guest operating system.
- Billing and payments are simulated. Monitoring and several infrastructure pages use stored or seeded records rather than live workload telemetry.
- The repository contains automated tests, but their outcomes must be recorded from an actual run in the team environment.
- Add the YouTube URL and database screenshots/results before final submission.

Future Work

- Connect one workflow to a local hypervisor, or expose the product clearly as a container-based simulator.
- If infrastructure integration is added, report provisioning stages and recoverable errors from actual operations.
- Collect monitoring data from real workloads and make stop/start behavior observable.
- Run the test suite against an isolated PostgreSQL test database and verify setup on a clean machine.

References

1. PostgreSQL 18 Documentation: https://www.postgresql.org/docs/18/

2. Node.js Documentation: https://nodejs.org/docs/latest/api/

3. React Documentation: https://react.dev/learn

4. Express 5.x API Reference: https://expressjs.com/en/5x/api.html

5. Amazon EC2 User Guide, "Troubleshoot an unreachable Amazon EC2 instance": https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/troubleshoot-unreachable-instance.html

6. OpenStack Dashboard User Documentation:
https://docs.openstack.org/horizon/latest/user/

7. OpenStack Horizon, “Launch and manage instances”:
https://docs.openstack.org/horizon/latest/user/launch-instances.html

8. DHARANEESWARAN08, CloudSphere GitHub repository:
https://github.com/DHARANEESWARAN08/CloudSphere

9. CloudSphere database migrations:
https://github.com/DHARANEESWARAN08/CloudSphere/tree/main/database/migrations